EFFICIENT ADAPTER TUNING FOR JOINT SINGING VOICE BEAT AND DOWNBEAT TRACKING WITH SELF-SUPERVISED LEARNING FEATURES

Jiajun Deng[1,2], Yaolong Ju[1], Jing Yang[1], Simon Lui[1], Xunying Liu[2]

[1] Huawei Technologies Co., Ltd., Shenzhen, China
[2] The Chinese University of Hong Kong, Hong Kong SAR, China
jjdeng@se.cuhk.edu.hk, yaolongju@huawei.com

ABSTRACT

Singing voice beat tracking is a challenging task, due to the lack of musical accompaniment that often contains robust rhythmic and harmonic patterns, something most existing beat tracking systems utilize and can be essential for estimating beats. In this paper, a novel temporal convolutional network-based beat-tracking approach featuring self-supervised learning (SSL) representations and adapter tuning is proposed to track the beat and downbeat of singing voices jointly. The SSL DistilHuBERT representations are utilized to capture the semantic information of singing voices and are further fused with the generic spectral features to facilitate beat estimation. Sources of variabilities that are particularly prominent with the non-homogeneous singing voice data are reduced by the efficient adapter tuning. Extensive experiments show that feature fusion and adapter tuning improve the performance individually, and the combination of both leads to significantly better performances than the un-adapted baseline system, with up to 31.6% and 42.4% absolute F1-score improvements on beat and downbeat tracking, respectively.

1. INTRODUCTION

Singing voice beat tracking is an important music information retrieval (MIR) task that can serve many downstream applications. For example, singing transcription can utilize beats to finetune the onsets of the transcribed notes for better accuracies [1] as well as automatic accompaniment generation, where the beat information can be instrumental for drum arrangements [2]. However, existing literature on beat tracking mostly focused on music with instrumental accompaniment [3–11], and tracking beats of singing voice is largely unaddressed and remains a key challenge to date. Its difficulty can be attributed to the lack of musical accompaniment that contains rhythmic and harmonic patterns vital for beat tracking in general. This leads to several challenges in developing effective singing voice beat tracking systems.

First, the existing state-of-the-art music beat tracking systems deliver poor performances on singing voices due to the notable inherent disparities between complete music songs and singing voices [12]. For example, the traditional music beat tracking system often learns latent mapping based on acoustic clues such as the spectrogram magnitude [13–15], which is often caused by the reoccurring drums or bass. Such clues, however, are barely present in singing voices. Inspired by the similarity between the singing voice and speech [16], the self-supervised learning (SSL) speech representations are utilized and demonstrate advantages over spectral features in singing voices [12].

Second, the naturalistic singing voice data is generally highly non-homogeneous due to its inherent variabilities from different conditions, such as genres, singers, recording devices, or languages [17]. The resulting high degree of singing voice heterogeneity may cause a large mismatch between training and test distributions, which can significantly degrade system performances. This issue is particularly prominent with the singing voice beat tracking that lacks musical accompaniment, as opposed to music beat tracking containing rich percussive and harmonic profiles.

To this end, we present a novel singing voice beat and downbeat tracking system using a temporal convolutional network featuring SSL representations and adapter tuning. More specifically, the SSL DistilHuBERT representations are utilized to capture the essential para-linguistics, semantic, and phonemic level characteristics and are further fused with the generic spectral features to facilitate beat estimations. A series of parameter-efficient adapters are performed to compensate for mismatch arising from the inherent variabilities among diverse singing voice datasets. The main contributions of the paper are summarized below:

1) To our knowledge, this paper is the first to investigate the joint beat and downbeat tracking task featuring the fusion of SSL representations and spectral features. In contrast, similar prior research was conducted in the context of only beat estimations [12] or beat/downbeat tracking using spectral features only [18].

2) Extensive experiments show that the train-test

© Jiajun Deng, Yaolong Ju, Jing Yang, Simon Lui, Xunying Liu. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** Jiajun Deng, Yaolong Ju, Jing Yang, Simon Lui, Xunying Liu, "Efficient Adapter Tuning for Joint Singing Voice Beat and Downbeat Tracking with Self-supervised Learning Features", in *Proc. of the 25th Int. Society for Music Information Retrieval Conf.,* San Francisco, United States, 2024.

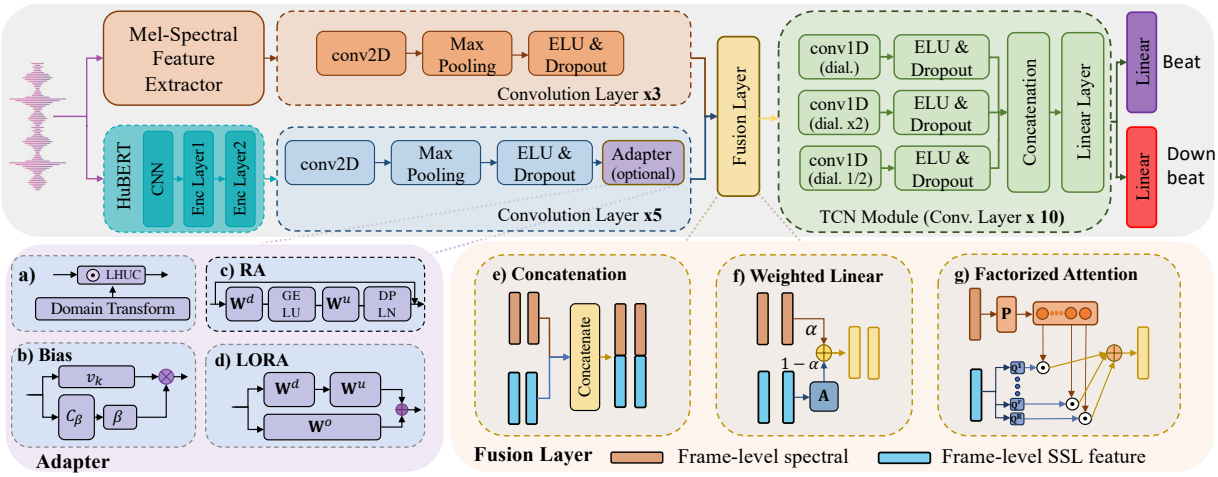


Figure 1. Examples of joint beat and downbeat tracking systems using Temporal Convolutional Network (TCN) shown in the light grey box (top). The pre-trained self-supervised DistilHuBERT model is shown in the deep cyan-blue box (top left). The parameter-efficient adapters described in Section 4 are shown in the light purple colored box (bottom left corner), which includes **a)** Learning Hidden Unit Contributions adaptation, **b)** Bias Adapter adaptation, **c)** Residual Adapter (RA) adaptation and **d)** Low-Rank Adaptation (LoRA). The fusion layer described in Section 3 is shown in the light orange box (bottom right), which includes **e)** Concatenation, **f)** Weighted linear interpolation fusion, and **g)** Factorized attention fusion.

data distribution mismatch issue presented in the non-homogeneous singing voice data significantly degrades the beat-tracking performance, particularly in downbeat estimation. To this end, inspired by the use of parameter-efficient adaptation techniques in machine learning fields [19–24], this paper presents the first work that successfully employs efficient adapter tuning approaches for singing voice beat-tracking tasks to address the mismatch above.

3) The efficacy of the proposed beat tracking approach is consistently demonstrated across various public datasets over the un-adapted baseline beat tracking system. In addition, the inherent generality of the proposed approach and the accompanying implementation details outlined in this paper allow their further application to other beat-tracking systems or MIR tasks.

2. TCN-BASED BEAT TRACKING SYSTEMS

In this paper, we adopt temporal convolutional network (TCN) as the backbone for singing voice beat tracking for two reasons: **1)** TCN has shown solid performances in the traditional beat tracking involving musical accompaniment. First proposed by [25], TCN achieved superior performances to the previous SOTA bi-directional LSTM and has been widely used for beat tracking since then [26, 27]. **2)** Although SpecTNT has recently outperformed TCN [9], TCN is still lightweight with way fewer parameters than SpecTNT, making it easy for deployment and cost-efficient as commercial applications.

2.1 Architecture

The conventional TCN-based beat tracking system consists of a front-end convolution module and a TCN module, connected by a fusion layer shown in Fig. 1 (light grey box, top). Each convolution layer in the front-end module has 20 channels, a stride of one, and kernels with various sizes. Max-pooling, exponential linear unit (ELU) activation [28], and dropout neural operations are applied to each convolution layer in sequence. The TCN module is stacked by ten dilated convolutional layers with exponentially increased dilation factors $2^0, 2^1, \cdots, 2^9$ resulting in a large receptive to capture long temporal contexts. Each dilated convolutional layer contains three dilated convolution blocks, each with different dilation rates (one dilation factor, half the dilation factor, and twice the dilation factor) and 20 channels. ELU activation and dropout operations are applied to each dilated convolution block, followed by an output linear layer shown in Fig. 1 (green box, top right).

2.2 Multi-task Learning

Based on the above TCN-based architecture, the beat-tracking task can be cast as a binary classification through time, for example, classifying the presence or absence of a beat for each frame. To perform the joint beat and downbeat tracking in a single system [26,27], an auxiliary downbeat tracking task by introducing a separate binary classification linear layer is carried out to produce the downbeat. Thus a multi-task criterion that interpolates between the beat and downbeat binary cross entropy (BCE) costs is adopted for training, which can be formulated as

$$\mathcal{L}_{MTL} = \eta \mathcal{L}_{BEAT} + (1-\eta)\mathcal{L}_{DBEAT}, \qquad (1)$$

where $\eta \in [0, 1]$ is the tunable hyper-parameter for balancing the beat BCE cost $\mathcal{L}_{BEAT}$ and downbeat BCE cost $\mathcal{L}_{DBEAT}$. Both beat and downbeat prediction outputs are post-processed with a dynamic Bayesian network (DBN) [29–31] to produce the final sequence of predictions [15].

2.3 Input Features

Two types of feature embeddings are fed into the TCN-based beat-tracking system. The first is the traditional 81-dim log-magnitude mel-frequency spectrogram features,

which are widely adopted in the music beat tracking tasks [9, 27, 32]. Another is the SSL DistilHuBERT features. DistilHuBERT [33, 34] is a lightweight self-supervised pre-trained speech foundation model. Its lighter architecture enables faster inference than other pre-trained foundation models. The 768-dim DistilHuBERT feature representations extracted from the last HuBERT layer are proven to serve as beneficial feature embeddings in analyzing singing rhythms [12] due to the acoustic and linguistic similarities between singing voices and speech.

3. FEATURE FUSION

The process of combining diverse feature representations, named feature fusion, plays a vital role in determining the effectiveness of beat-tracking systems [35]. To this end, several fusion approaches are introduced in this section to integrate the traditional spectrogram and pre-trained HuBERT feature representations effectively.

3.1 Early Feature Fusion

Early feature fusion is the combination of diverse feature representations performed early in a neural network [36]. For example, the features are fused at the network's input layer. Let $\mathbf{x} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_T] \in \mathbb{R}^{m\times T}$ and $\mathbf{u} = [\boldsymbol{u}_1, \boldsymbol{u}_2, \cdots, \boldsymbol{u}_T] \in \mathbb{R}^{n\times T}$ denote the spectrogram and HuBERT feature representations with T frames, respectively. Two forms of early feature fusion are investigated.

a) Input concatenation refers to directly concatenating the spectrogram and HuBERT features at the frame level. The concatenated feature representation $\mathbf{z}$ at t-th frame can be expressed as $\boldsymbol{z}_t = [\boldsymbol{x}_t; \boldsymbol{u}_t] \in \mathbb{R}^{n+m}$.

b) Weighted linear interpolation refers to interpolating the frame-level spectrogram and HuBERT feature representations with a learnable hyper-parameter $\alpha \in [0, 1]$. The interpolated feature representation at t-th frame can be formulated as $\boldsymbol{z}_t = \alpha\boldsymbol{x}_t + (1-\alpha)\mathbf{A}\boldsymbol{u}_t$, where $\mathbf{A} \in \mathbb{R}^{n\times m}$ is a learnable projection matrix to enable the dimension of HuBERT features to be consistent with that of the spectrogram features.

3.2 Late Feature Fusion

The combination of diverse features at a later network layer leads to late feature fusion [37]. This allows the model to leverage high-level, abstract representations, leading to more informed decisions and improved performance. As shown in Fig. 1, the spectrogram and pre-trained HuBERT features are first fed into a separate CNN module before being further combined using different fusion schemes. Let $\hat{\mathbf{x}} = [\hat{\boldsymbol{x}}_1, \hat{\boldsymbol{x}}_2, \cdots, \hat{\boldsymbol{x}}_T] \in \mathbb{R}^{k\times T}$ and $\hat{\mathbf{u}} = [\hat{\boldsymbol{u}}_1, \hat{\boldsymbol{u}}_2, \cdots, \hat{\boldsymbol{u}}_T] \in \mathbb{R}^{k\times T}$ denote the high-level CNN output hidden representations with T frames using the spectrogram and HuBERT features, respectively. The concatenation and weighted combination operations can also be performed in a late fusion style, which is illustrated as **a)** late concatenation fusion $\hat{\boldsymbol{z}}_t = [\hat{\boldsymbol{x}}_t; \hat{\boldsymbol{u}}_t] \in \mathbb{R}^{2k}$ and **b)** late weighted linear interpolation fusion $\hat{\boldsymbol{z}}_t = \alpha\hat{\boldsymbol{x}}_t + (1-\alpha)\hat{\boldsymbol{u}}_t$.

3.3 Factorized Attention Fusion

In order to focus on relevant selective representations while suppressing less important ones, factorized attention fusion is performed in a late fusion fashion. The SSL hidden representation at t-th frame $\hat{\boldsymbol{u}}_t$ are first factorized into R subspace representations $[\boldsymbol{v}_t^1, \boldsymbol{v}_t^2, \cdots, \boldsymbol{v}_t^R] \in \mathbb{R}^{k\times R}$ using a series of parallel linear transforms, which is expressed as

$$[\boldsymbol{v}_t^1, \boldsymbol{v}_t^2, \cdots, \boldsymbol{v}_t^R] = [\mathbf{Q}^1, \mathbf{Q}^2, \cdots, \mathbf{Q}_t^R]\hat{\boldsymbol{u}}_t, \tag{2}$$

where $\mathbf{Q}^r \in \mathbb{R}^{k\times k}$ is the linear transformations for r-th subspace. The spectrogram hidden embedding at t-th frame $\hat{\boldsymbol{x}}_t$ is projected into a R-dim interpolation vector $\boldsymbol{w}_t = [w_t^1, w_t^2, \cdots, w_t^R] \in \mathbb{R}^R$ using a projection matrix $\mathbf{P} \in \mathbb{R}^{R\times k}$, which is given as $\boldsymbol{w}_t = \text{Softmax}(\mathbf{P}\hat{\boldsymbol{x}}_t)$. Subsequently, the fused feature representation can be obtained by an attention mechanism [38],

$$\hat{\boldsymbol{z}}_t = \text{Sigmoid}(\sum_{r=1}^{R} w_t^r \boldsymbol{v}_t^r). \tag{3}$$

4. PARAMETER EFFICIENT ADAPTATION

A straightforward solution to reduce the mismatch between training and evaluation distributions is to directly fine-tune the entire system using the target-domain singing voice data. However, this adaptation scheme not only encounters overfitting problems due to the scarcity of singing voices but also poses key challenges to adaptation parameter storage. Parameter-efficient adaptation approaches [39–41] that introduce limited adaptation parameters with the original model parameters unchanged have been proposed to tail for the above overfitting and parameter storage issues. Inspired by this idea, several prominent parameter-efficient adapters are explored for singing voice beat-tracking systems in this section.

4.1 Learning Hidden Unit Contributions

Learning hidden unit contributions (LHUC) adaptation is an effective speaker adaptation solution that accounts for speaker variation of speech [42]. The basic idea of LHUC adaptation is to modify the amplitudes of activation outputs using a scaling vector. Let $\boldsymbol{r}_{l,e} \in \mathbb{R}^u$ denote the adaptation parameters for the e-th domain in the l-th hidden layer, where u is the dimension of adaptation parameters. The adapted hidden output $\boldsymbol{h}_{l,k}$ can be given as

$$\boldsymbol{h}_{l,k} = \boldsymbol{h}_l \odot \xi(\boldsymbol{r}_{l,k}), \tag{4}$$

where $\boldsymbol{h}_l$ is the original hidden activation output at the l-th hidden layer, $\odot$ is the Hadamard product operation, $\xi(\boldsymbol{r}_{l,e})$ is the scaling vector parameterized by $\boldsymbol{r}_{l,e}$, and $\xi(\cdot)$ is the element-wise $2\times\text{Sigmoid}(\cdot)$ function with a range of $(0, 2)$. During adaptation, the adaptation parameters $\boldsymbol{r}_{l,k}$ for each domain are initialized as zeros vector. An example of LHUC adaptation is shown in Fig. 1(a).

4.2 Bias Adaptation

The bias adapter adaptation [43] adds frame-level bias to the hidden representation shifts using a domain-dependent shift vector $\boldsymbol{v}_e \in \mathbb{R}^u$ and a linear layer C_β, which is shown in Fig. 1(b). The frames crucial for beat tracking should be assigned a larger representation shift compared to other frames. The linear layer produces a frame-level weight vector $\boldsymbol{\beta} = C_\beta \boldsymbol{h}_l = [\beta_1, \beta_2, \cdots, \beta_T] \in \mathbb{R}^T$, where β_t denotes the weight of the t-th frame hidden representation. Therefore, the domain-dependent representation shifts $\boldsymbol{v}_e$ can be enhanced by applying frame-level weights, and the adapted hidden layer output can be expressed as

$$\boldsymbol{h}_{l,e} = \boldsymbol{h}_l + \boldsymbol{v}_e \otimes \beta, \tag{5}$$

where $\otimes$ is the outer product operation, and the outer product of shift-vector $\boldsymbol{v}$ and the weight β can be expressed as $\boldsymbol{v}_e \otimes \beta = [\beta_1 \boldsymbol{v}_e, \beta_2 \boldsymbol{v}_e, \cdots, \beta_T \boldsymbol{v}_e] \in \mathbb{R}^{u \times T}$.

4.3 Residual Adapter

Inspired by the residual idea [44], a residual adapter (RA) with slight modifications is designed for beat tracking. The adapter starts with a down-linear projection $\mathbf{W}_e^d \in \mathbb{R}^{r \times u}$, followed by a non-linear GeLU activation function $\zeta(\cdot)$, and an up-linear projection $\mathbf{W}_e^u \in \mathbb{R}^{u \times r}$. Let $f_{RA}(\cdot; \boldsymbol{\Theta}_{l,e})$ denote the residual adapter function for e-th domain in the l-th hidden layer, where $\boldsymbol{\Theta}_{l,e}$ is the adaptation parameters for e-th domain. The adapted hidden outputs are given as

$$\boldsymbol{h}_{l,k} = \boldsymbol{h}_l + f_{RA}(\boldsymbol{h}_l; \boldsymbol{\Theta}_{l,e}), \tag{6}$$

$$f_{RA}(\boldsymbol{h}_l; \boldsymbol{\Theta}_{l,e}) = \mathrm{LN}(\mathrm{DP}(\mathbf{W}_{l,e}^u \zeta(\mathbf{W}_{l,e}^d \boldsymbol{h}_l))), \tag{7}$$

where $\mathrm{DP}(\cdot)$ and $\mathrm{LN}(\cdot)$ denote the dropout and layernorm operations, respectively. The adaptation capacity can be controlled by managing the number of parameters in each adapter module through controlling the bottleneck dimension r. An example of an RA adapter is shown in Fig. 1(c).

4.4 Low-rank Adaptation

Instead of the non-parallel nature of adapter modules that consumes additional GPU time mentioned above, Low-rank adaptation (LoRA) [45] reduces the number of adaptation parameters by learning rank-decomposition matrix pairs $\{\mathbf{W}^d, \mathbf{W}^u\}$ while freezing the original weights. The LoRA-adapted linear hidden output can be expressed as

$$\boldsymbol{h}_{l,k} = f_{LoRA}(\boldsymbol{h}_{l-1}; \boldsymbol{\Theta}_{l,e}), \tag{8}$$

$$= (\mathbf{W}_l^o + \mathbf{W}_{l,e}^u \mathbf{W}_{l,e}^d) \boldsymbol{h}_{l-1}, \tag{9}$$

where $f_{LoRA}(\cdot; \boldsymbol{\Theta}_{l,e})$ is the LoRA adapter, $\mathbf{W}_l^o \in \mathbb{R}^{n \times u}$ is the original pre-trained weight matrix, $\mathbf{W}_{l,e}^d \in \mathbb{R}^{r \times u}$ and $\mathbf{W}_{l,e}^u \in \mathbb{R}^{n \times r}$ are the trainable low-rank decomposition matrices. It is noted that the rank $r \ll \min(u, n)$ is far less than the dimension of the original matrix, which allows for reducing the number of adaptation parameters. An example of a LoRA adapter is shown in Fig. 1(d).

Table 1. Description of the singing voice beat tracking datasets. † and $*$ represent the music beat tracking dataset and the music source separation dataset, respectively.

Dataset	# Hours	# Excerpts	Genres
GTZAN† [48]	5.9	754	Blues, Country, Disco, Hiphop, etc.
RWC Pop† [49]	5.4	273	Japanese Pop., etc.
Ballroom† [50]	2.8	313	Rumba, Tango, Waltz, Jive, etc.
Hainsworth† [51]	1.9	173	Jazz, Metal, Rock, Opera, etc.
MUSDB18* [52]	6.4	144	Pop., Country, Rock, etc.
URSing* [53]	3.4	142	Chinese Pop., etc.

4.5 Estimation of Adaptation Parameters

Let $\boldsymbol{\mathcal{D}}_e = \{\boldsymbol{X}_e, \boldsymbol{Y}_e\}$ denote the adaptation data for e-th domain, where $\boldsymbol{X}_e$ and $\boldsymbol{Y}_e$ stand for the singing voice waveform and the corresponding beat/downbeat sequences, respectively. Without loss of generality and for simplicity, let $\boldsymbol{\Theta}$ denote the original model parameters. In the context of adaptation, the adaptation parameters $\boldsymbol{\Theta}_e$ conditioned on the e-th domain can be estimated by minimizing the loss function in Eqn. (1), which is given by

$$\hat{\boldsymbol{\Theta}}_e = \arg\min_{\boldsymbol{\Theta}_e} \{\mathcal{L}_{MTL}(\boldsymbol{\mathcal{D}}_e; \boldsymbol{\Theta}, \boldsymbol{\Theta}_e)\}. \tag{10}$$

5. EXPERIMENTS

5.1 Datasets and Evaluation Metrics

To the best of our knowledge, there are no publicly available datasets that include pristine vocal audio alongside beats and downbeats annotations. Annotating beats and downbeats based solely on vocal signals can be arduous and subjective, even by human experts, since there are no evident rhythmic cues like percussive instruments to accurately comprehend the singer's rhythmic intentions.

Therefore, we follow the strategy described in [12] to utilize the existing public MIR datasets and systems to create the singing voice data with beat/downbeat annotations. This includes **a) four music beat tracking datasets** with available beat annotations, where the singing signals are extracted by the Demucs source separation model [46], and **b) two music source separation datasets** with available isolated singing tracks, where the preliminary beats and downbeats annotations are generated by the existing TCN-based beat tracking system [25] using the full music mix (singing with musical accompaniment), then manual revision is further performed to correct the potential annotation errors. Altogether, six datasets are used in this paper as shown in Table 1, where a silence-stripping technique is applied to each dataset to remove the long chunks of silence. The 90% of the whole data randomly selected from a uniform distribution is used for training, while the remaining 10% is used for evaluation. The evaluation metric of F1-score with a tolerance window of ± 70 ms, a typical setting commonly used in the traditional beat tracking [25], is adopted for our performance evaluation. We also adopt P-score, Cemgil, and Goto [47] as additional evaluation metrics to further demonstrate the advantages of the proposed approaches in the final experiments (Table 4).

Table 2. Beat and downbeat tracking performance of TCN systems using different feature fusion methods evaluated on the GTZAN, RWC pop (RWCPO), and MUSDB18 (MUSDB) datasets in terms of F1-score.

ID	Input Features	Fusion Methods	GTZAN	RWCPO	MUSDB
Beat/Downbeat Tracking F1 Scores					
1	Spectrogram	-	0.48/0.26	0.65/0.53	0.31/0.15
2	DistilHuBERT	-	0.74/0.47	0.76/0.68	0.38/0.17
3	Spectogram & DistilHuBERT	Input Concatenation	0.78/0.56	0.83/0.79	0.41/0.25
4		Input Weighted	0.77/0.55	0.86/0.81	0.43/0.25
5		Late Concatenation	**0.81**/0.53	0.87/0.81	0.45/0.25
6		Late Weighted	0.79/**0.58**	0.88/**0.84**	0.47/**0.26**
7		Factorized Attention	0.80/0.51	**0.91**/0.82	**0.48**/0.23

5.2 Implementation Details

Two feature extractors, including the mel-spectrogram feature extractor and the pre-trained SSL DistilHuBERT feature extractor [34], are employed to generate the 81-dimensional spectral features and 768-dimensional SSL feature representations of vocal signals. In this paper, the vocal signals of all datasets are resampled to 16000 Hz. As illustrated in Section 2, the temporal convolution network consists of a front-end convolution and TCN modules. The front-end convolution module tailored for late feature fusion consists of a 3-layer convolution network[1] and a 5-layer convolution network[2] for processing the spectral and SSL feature representations, respectively. The kernel size and stride for the Max-pooling operation are 1×3 for all convolution layers. The TCN module consists of ten dilated convolution layers, wherein the dilation factors increase exponentially.

During the TCN-based beat tracking system training, all weights of the system are randomly initialized. The Ranger optimizer [54] with an initial learning rate of 0.001, the ReduceLROnPlateau scheduler with a factor of 0.9 and patience of 5, and a dropout rate of 0.1 are used for training and adaptation. The training and adaptation epochs are set as 100 and 30, respectively. The hyper-parameter of multi-task learning in Eqn. (1) is empirically set as $\eta = 0.2$. Since the ratio of positive and negative examples in the beat-tracing task is often imbalanced, the weighted binary cross-entropy loss is applied, and the weights of positive examples for the beat and downbeat costs are set to be 10 and 20, respectively.

5.3 Performance of Feature Fusion

Table 2 shows the beat and downbeat tracking F1 scores of TCN systems using different feature representation fusion methods. Several important findings can be observed. **a)** The systems using SSL DistilHuBERT features (ID.2) show better F1 performance than those using the traditional spectral features (ID.1) in all three evaluation sets. This demonstrates that the semantic information captured by SSL speech representation is crucial for singing voice beat tracking. **b)** All systems that leverage feature fusion approaches (ID.3-7) outperform the systems using only one single spectral (ID.1) or SSL feature (ID.2). This confirms our motivation that spectral and SSL features are complementary as they capture different characteristics of vocal signals. **c)** The late weighted linear interpolation fusion method (ID.6) achieves the best F1 results on the downbeat tracking and competitive beat tracking performance among all fusion approaches, therefore we selected it for the following experiments.

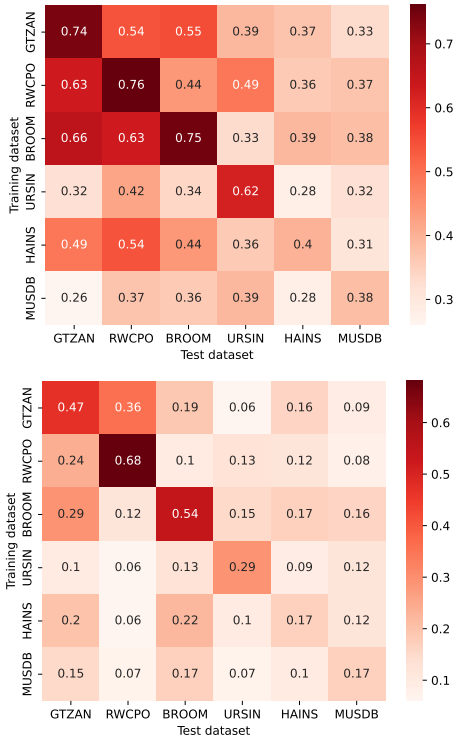


Figure 2. F1-score of the beat (upper) and downbeat (bottom) tracking when training data and test data are collected from the same (diagonal value) or different sources.

5.4 Performance of Adaptation

The mismatch across different datasets is revealed in Fig. 2. **a)** When the system is trained on singing voice data from the same source as the test data, the best beat and downbeat tracking performance are obtained (diagonal value). **b)** The mismatch between training and test distributions (non-diagonal) significantly degrades the beat tracking performance, especially in downbeat tracking. This confirms our assumption that the mismatch across different datasets is particularly prominent in singing voice beat tracking due to the lack of robust rhythmic and harmonic patterns. Therefore, mismatch is an essential issue that needs to be addressed in multi-condition training.

Table 3 shows the beat and downbeat tracking performance of the TCN systems configured with different adapters using only DistilHuBERT features. Several trends can be found. **a)** It is not surprising that multi-condition systems (ID.2) trained on all six datasets do not always outperform the in-domain systems (ID.1) trained on the well-controlled data from the same source as test data because of the mismatch issue. This demonstrates that

[1] The channel, kernel size, stride, and padding of Conv2D used in 3-layer convolution network for each convolution layer are {20, 20, 20}, {3x3,1x12,1x3}, {1,1,1} and {1x0, 0x0, 1x0} respectively.

[2] The channel, kernel size, stride, and padding of Conv2D used in 5-layer convolution network for each convolution layer are {20, 20, 20, 20, 20}, {3x3,1x12,1x3,3x3,1x12,3x3}, {1,1,1,1,1} and {1x0, 0x0, 1x0,0x0,1x0} respectively.

Table 3. Beat and downbeat tracking performance of TCN systems configured with different adapters on the GTZAN, RWCPO, Ballroom (BROOM), Hainsworth (HAINS), MUSDB, and URSing (URSIN) datasets in terms of F1-score.

ID	Systems	Adapter			Datasets					
		Method	Location	# Params	GTZAN	RWCPO	BROOM	HAINS	MUSDB	URSIN
Beat/Downbeat Tracking F1-scores										
1	In-domain	-	-	-	0.74/0.47	0.76/0.68	0.75/0.54	0.40/0.17	0.38/0.17	0.62/0.29
2	Multi-condition	-	-	-	0.78/0.57	0.89/0.45	0.72/0.43	0.50/0.33	0.53/0.25	0.61/0.22
3	Multi-condition with Adaptation	Fine-tune	ALL Layers	100%	**0.80**/0.59	**0.91/0.82**	**0.80/0.57**	0.54/0.38	0.55/0.38	**0.72**/0.34
4		LHUC	First CNN Layer	5%	0.78/0.56	0.88/0.59	0.72/0.46	0.50/0.33	0.53/0.26	0.62/0.25
5		BIAS	First CNN Layer	10%	0.79/0.57	0.89/0.61	0.71/0.47	0.52/0.35	0.55/0.31	0.62/0.24
6		LoRA	First CNN Layer	20%	**0.80**/0.62	0.90/0.68	0.78/0.56	0.54/0.37	0.56/0.38	0.66/0.33
7		RA	First CNN Layer	20%	**0.80/0.65**	**0.91**/0.80	**0.80/0.57**	**0.58/0.43**	**0.58/0.41**	0.68/**0.38**
8		RA	Second CNN Layer	10%	0.78/0.60	0.90/0.77	0.76/0.56	0.57/0.41	0.55/0.40	0.68/0.35
9		RA	Third CNN Layer	5%	0.79/0.62	0.90/0.78	0.76/0.56	0.57/0.39	0.55/0.40	0.68/0.33

Table 4. Beat and downbeat tracking performance of the proposed TCN systems incorporating fusion and adapters.

ID	Systems	Input Features	Feature Fusion	Adapter	Beat Tracking				Downbeat Tracking			
					F1	P-score	Cemgil	Goto	F1	P-score	Cemgil	Goto
1	Multi-condition	Spectrogram	✗	✗	0.497	0.541	0.410	0.429	0.254	0.420	0.212	0.256
2	Multi-condition	DistilHuBERT	✗	✗	0.656	0.681	0.565	0.561	0.389	0.501	0.351	0.370
3	Proposed	Spec. & HuBERT	✓	✗	0.774	0.756	0.684	0.703	0.524	0.603	0.487	0.520
4	Proposed	Spec. & HuBERT	✓	✓	**0.813**	**0.801**	**0.713**	**0.757**	**0.678**	**0.692**	**0.621**	**0.663**

blindly expanding the training data is insufficient to enhance the system's generalization. **b)** All systems configured with adapters (ID.4-7) improve the performance over both multi-condition systems (ID.2) and in-domain systems (ID.1), which suggests that parameter-efficient adapter tuning methods can address the mismatch issue effectively. **c)** The residual adapter (RA) (ID.7) applied at the first CNN layer achieves the best results relative to other adaptation approaches. It is noteworthy that RA adaptation using only 20% of adaptation parameters shows comparable performance to fully fine-tuned techniques (ID.3). In addition, the observation that the performance gain of downbeat tracking is greater than that of beat tracking is consistent with our finding in Fig. 2 that the downbeat tracking performance is more sensitive to the mismatch issue. **d)** When incorporating adapters into the second or third CNN layer (ID.8,9), with acceptable performance degradation, RA adaptation delivers a much lighter architecture with fewer parameters.

5.5 Performance of The Proposed Method

The advantages of the proposed method incorporating both late weighted linear interpolation feature fusion and RA adapter are demonstrated in Table 4. The evaluation results are the overall performance of all six evaluation sets using micro averaging. Two main observations can be found.

First, the multi-condition system using the proposed feature fusion approaches (ID.3) still outperforms the systems (ID.1,2) using only one spectral or SSL feature. Of particular interest, this system (ID.3) is compared to the existing singing voice beat tracking system [12], where the same evaluation protocol is followed by using the entire 5.9-hrs GTZAN dataset for testing[3]. As a result, our system achieved beat tracking F1-score of 0.784 on GTZAN, a significant **5.1%** absolute improvement compared to 0.733 from [12] even using less training data.

Second, consistent performance improvements across all evaluation metrics are observed when the adapter tuning scheme (ID.4) is applied. Overall significant F1-score improvements of up to **31.6%** and **42.4%** absolute were obtained over the baseline un-adapted system using only one feature on the beat and downbeat tracking, respectively. In particular, the beat/downbeat performances of $0.87/0.78$, $0.95/0.87$, $0.85/0.75$, $0.66/0.49$, $0.68/0.49$, and $0.79/0.41$ are achieved by the proposed approach (ID.4) on the test split of GTZAN, RWC pop, Ballroom, Hainsworth, MUSDB, and URSing datasets, respectively.

6. CONCLUSIONS

This paper proposed a temporal convolution network based beat-tracking framework featuring self-supervised learning (SSL) representations and efficient adapter tuning to track the beat and downbeat of singing voices jointly. Feature fusion strategies were performed to leverage the advantages of the generic spectral and SSL speech feature representations. Efficient adapter tuning was utilized to mitigate the sources of variabilities of the non-homogeneous singing voice data. Experimental results showed that the proposed approach significantly outperforms the un-adapted baseline system using only spectral or SSL features. The inherent generality of the proposed approaches allows their further application to other beat-tracking systems or MIR tasks. Future work will focus on solving the data sparsity issue of the singing voice beat tracking task.

7. ACKNOWLEDGMENTS

This research is supported by Hong Kong RGC GRF grant No. 14200021, 14200220, Innovation & Technology Fund grant No. ITS/254/19 and ITS/218/21. Jiajun Deng carried out this research during his internship at Huawei Hong Kong Research Center. The authors would like to thank Huawei for its hardware and software services.

[3] The remaining five datasets are therefore used for training our system, which is less data compared to [12].

8. REFERENCES

[1] R. Nishikimi, E. Nakamura, M. Goto, and K. Yoshii, "Audio-to-score singing transcription based on a crnn-hsmm hybrid model," *APSIPA Transactions on Signal and Information Processing*, vol. 10, p. 7, 2021.

[2] Y.-K. Wu, C.-Y. Chiu, and Y.-H. Yang, "Jukedrummer: Conditional Beat-aware Audio-domain Drum Accompaniment Generation via Transformer VQ- VAE," in *International Society for Music Information Retrieval Conference (ISMIR)*, 2022, pp. 193–200.

[3] P. E. Allen and R. B. Dannenberg, "Tracking musical beats in real time," in *ICMC*, 1990.

[4] M. E. Davies and M. D. Plumbley, "Context-dependent beat tracking of musical audio," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 15, no. 3, pp. 1009–1020, 2007.

[5] A. Mottaghi, K. Behdin, A. Esmaeili, M. Heydari, and F. Marvasti, "Obtain: Real-time beat tracking in audio signals," *International Journal of Signal Processing Systems*, 2017.

[6] Y.-T. Wu, Y.-J. Luo, T.-P. Chen, I. Wei, J.-Y. Hsu, Y.-C. Chuang, L. Su *et al.*, "Omnizart: A general toolbox for automatic music transcription," *The Journal of Open Source Software*, vol. 6, no. 68, p. 3391, 2021.

[7] B. Di Giorgi, M. Mauch, and M. Levy, "Downbeat tracking with tempo-invariant convolutional neural networks," *International Society for Music Information Retrieval Conference (ISMIR)*, 2021.

[8] M. Goto and Y. Muraoka, "Musical understanding at the beat level: real-time beat tracking for audio signals," in *Computational Auditory Scene Analysis*. CRC Press, 2021, pp. 157–176.

[9] Y.-N. Hung, J.-C. Wang, X. Song, W.-T. Lu, and M. Won, "Modeling beats and downbeats with a time-frequency transformer," in *International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2022, pp. 401–405.

[10] C. Hernandez-Olivan and J. R. Beltran, "Music composition with deep learning: A review," *Advances in Speech and Music Technology: Computational Aspects and Applications*, pp. 25–50, 2022.

[11] M. Won, Y.-N. Hung, and D. Le, "A foundation model for music informatics," in *International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 1226–1230.

[12] M. Heydari and Z. Duan, "Singing beat tracking with self-supervised front-end and linear transformers," *International Society for Music Information Retrieval Conference (ISMIR)*, 2022.

[13] J. L. Oliveira, F. Gouyon, L. G. Martins, and L. P. Reis, "Ibt: A real-time tempo and beat tracking system." in *International Society for Music Information Retrieval Conference (ISMIR)*, 2010, pp. 291–296.

[14] A. Gkiokas, V. Katsouros, G. Carayannis, and T. Stajylakis, "Music tempo estimation and beat tracking by applying source separation and metrical relations," in *International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2012, pp. 421–424.

[15] S. Böck, F. Krebs, and G. Widmer, "Joint beat and downbeat tracking with recurrent neural networks." in *International Society for Music Information Retrieval Conference (ISMIR)*, 2016, pp. 255–261.

[16] B. Lindblom and J. Sundberg, "The human voice in speech and singing," *Springer Handbook of Acoustics*, pp. 703–746, 2014.

[17] M. Bunch and M. Bunch, *Dynamics of the singing voice*. Springer, 1982.

[18] M. Heydari, J.-C. Wang, and Z. Duan, "Singnet: a real-time singing voice beat and downbeat tracking system," in *International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.

[19] A. Søgaard, *Semi-supervised learning and domain adaptation in natural language processing*. Springer Nature, 2022.

[20] Y. Wang, S. Agarwal, S. Mukherjee, X. Liu, J. Gao, A. H. Awadallah, and J. Gao, "Adamix: Mixture-of-adaptations for parameter-efficient model tuning," *Empirical Methods in Natural Language Processing (EMNLP)*, 2022.

[21] N. Ding, Y. Qin, G. Yang, F. Wei, Z. Yang, Y. Su, S. Hu, Y. Chen, C.-M. Chan, W. Chen *et al.*, "Parameter-efficient fine-tuning of large-scale pre-trained language models," *Nature Machine Intelligence*, vol. 5, no. 3, pp. 220–235, 2023.

[22] P. Bell, J. Fainberg, O. Klejch, J. Li, S. Renals, and P. Swietojanski, "Adaptation algorithms for neural network-based speech recognition: An overview," *Open Journal of Signal Processing*, vol. 2, pp. 33–66, 2020.

[23] B. Li, D. Hwang, Z. Huo, J. Bai, G. Prakash, T. N. Sainath, K. C. Sim, Y. Zhang, W. Han, T. Strohman *et al.*, "Efficient domain adaptation for speech foundation models," in *International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.

[24] K. C. Sim, Z. Huo, T. Munkhdalai, N. Siddhartha, A. Stooke, Z. Meng, B. Li, and T. Sainath, "A comparison of parameter-efficient asr domain adaptation methods for universal speech and language models,"

in *International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 6900–6904.

[25] E. MatthewDavies and S. Böck, "Temporal convolutional networks for musical audio beat tracking," in *European Signal Processing Conference (EUSIPCO)*. IEEE, 2019, pp. 1–5.

[26] S. Böck, M. E. Davies, and P. Knees, "Multi-task learning of tempo and beat: Learning one to improve the other." in *International Society for Music Information Retrieval Conference (ISMIR)*, 2019, pp. 486–493.

[27] S. Böck and M. E. Davies, "Deconstruct, analyse, reconstruct: How to improve tempo, beat, and downbeat estimation." in *International Society for Music Information Retrieval Conference (ISMIR)*, 2020, pp. 574–582.

[28] S. Singh, "Elu as an activation function in neural networks," *Deep Learning University*, 2020.

[29] K. P. Murphy *et al.*, "Dynamic bayesian networks," *Probabilistic Graphical Models, M. Jordan*, vol. 7, p. 431, 2002.

[30] D. P. Ellis, "Beat tracking by dynamic programming," *Journal of New Music Research*, vol. 36, no. 1, pp. 51–60, 2007.

[31] F. Krebs, S. Böck, and G. Widmer, "An efficient state-space model for joint tempo and meter tracking." in *International Society for Music Information Retrieval Conference (ISMIR)*, 2015, pp. 72–78.

[32] J. R. Zapata, M. E. Davies, and E. Gómez, "Multi-feature beat tracking," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 4, pp. 816–825, 2014.

[33] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "Hubert: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021.

[34] H.-J. Chang, S.-w. Yang, and H.-y. Lee, "Distilhubert: Speech representation learning by layer-wise distillation of hidden-unit bert," in *International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2022, pp. 7087–7091.

[35] M. Heydari, F. Cwitkowitz, and Z. Duan, "Beatnet: Crnn and particle filtering for online joint beat downbeat and meter tracking," *International Society for Music Information Retrieval Conference (ISMIR)*, 2021.

[36] K. Gadzicki, R. Khamsehashari, and C. Zetzsche, "Early vs late fusion in multimodal convolutional neural networks," in *International Conference on Information Fusion (FUSION)*. IEEE, 2020, pp. 1–6.

[37] N. Mungoli, "Adaptive feature fusion: Enhancing generalization in deep learning models," *arXiv preprint arXiv:2304.03290*, 2023.

[38] Y. Dai, F. Gieseke, S. Oehmcke, Y. Wu, and K. Barnard, "Attentional feature fusion," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2021, pp. 3560–3569.

[39] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for nlp," in *International Conference on Machine Learning (ICML)*, 2019, pp. 2790–2799.

[40] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick, and G. Neubig, "Towards a unified view of parameter-efficient transfer learning," *International Conference on Learning Representations (ICLR)*, 2021.

[41] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. A. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 1950–1965, 2022.

[42] P. Swietojanski, J. Li, and S. Renals, "Learning hidden unit contributions for unsupervised acoustic model adaptation," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 24, no. 8, pp. 1450–1463, 2016.

[43] C.-L. Fu, Z.-C. Chen, Y.-R. Lee, and H.-y. Lee, "Adapterbias: Parameter-efficient token-dependent representation shift for adapters in nlp tasks," *Findings of the Association for Computational Linguistics (ACL)*, 2022.

[44] K. Tomanek, V. Zayats, D. Padfield, K. Vaillancourt, and F. Biadsy, "Residual adapters for parameter-efficient asr adaptation to atypical and accented speech," *Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[45] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," *International Conference on Learning Representations (ICLR)*, 2021.

[46] A. Défossez, "Hybrid spectrogram and waveform source separation," *ISMIR Workshop on Music Source Separation*, 2021.

[47] M. E. Davies, N. Degara, and M. D. Plumbley, "Evaluation methods for musical audio beat tracking algorithms," *Queen Mary University of London, Centre for Digital Music, Tech. Rep. C4DM-TR-09-06*, 2009.

[48] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002.

[49] T. De Clercq and D. Temperley, "A corpus analysis of rock harmony," *Popular Music*, vol. 30, no. 1, pp. 47–70, 2011.

[50] F. Gouyon, A. Klapuri, S. Dixon, M. Alonso, G. Tzanetakis, C. Uhle, and P. Cano, "An experimental comparison of audio tempo induction algorithms," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 14, no. 5, pp. 1832–1844, 2006.

[51] S. W. Hainsworth and M. D. Macleod, "Particle filtering applied to musical tempo tracking," *Journal on Advances in Signal Processing*, vol. 2004, pp. 1–11, 2004.

[52] Z. Rafii, A. Liutkus, F.-R. Stöter, S. I. Mimilakis, and R. Bittner, "Musdb18-a corpus for music separation," *Available: https: //doi.org/10.5281/zenodo.1117372*, 2017.

[53] B. Li, Y. Wang, and Z. Duan, "Audiovisual singing voice separation." *Transactions of the International Society for Music Information Retrieval*, vol. 4, no. 1, pp. 195–210, 2021.

[54] L. Wright and N. Demeure, "Ranger21: a synergistic deep learning optimizer," *arXiv preprint arXiv:2106.13731*, 2021.